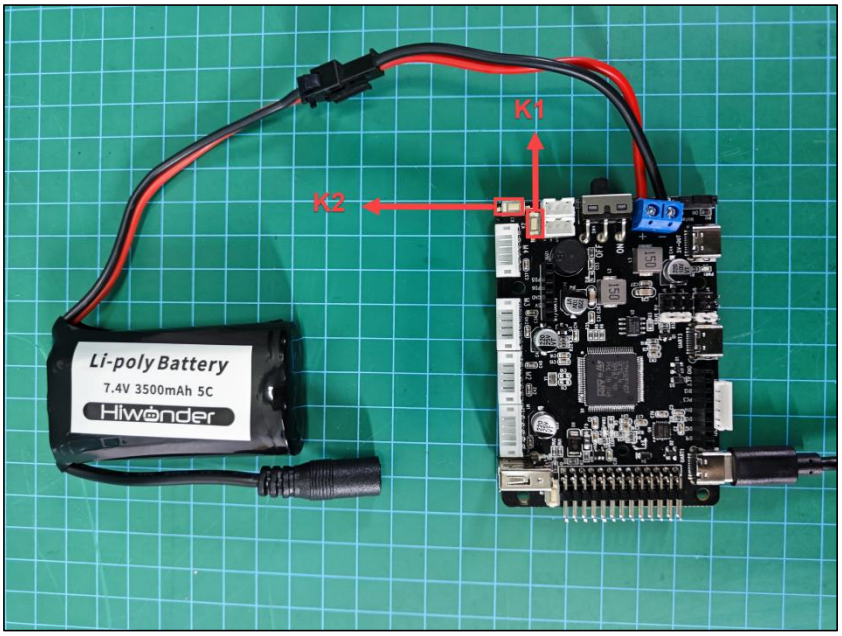
Lesson 3 RRC Button Detection Source Code Explanation

1. Program Logic

This chapter mainly focuses on controlling the buzzer sound through the built-in buttons on the STM32 main control board, accompanied by different LED status blinking.

2. Hardware Connection

The hardware connection diagram is as below:



3. Program Download

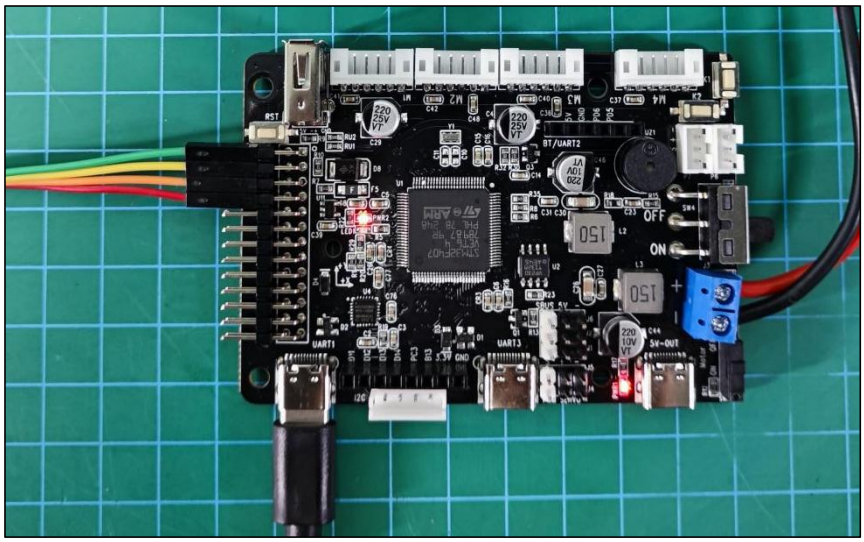
The example for this section is '**rosrobotcontrollerm4_key_buzzer**'.

For specific instructions on how to download the program, refer to the document: '**2. STM32 Development Fundamentals/Lesson 3 Project**

Compilation and Download'.

4. Program Outcome

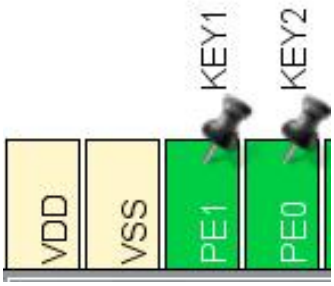
1. Press the K2 button twice at startup, and the buzzer will emit three consecutive beeps.
2. Press the K1 button; upon a single press, the buzzer emits 3 beeps. After the first press, the LED turns off; after the second press, the LED turns on; after the third press, the LED starts blinking.



5. PIN Configuration

Using the STM32CubeMX software to check the pin information, according to the diagram below, it can be seen that the chip pin corresponding to the buzzer is PA8. The pin configuration corresponding to the buzzer is PE1 and PE0.





6. Source Code Analysis

The source code for this lesson is located in the same folder as this tutorial document, primarily analyzing the files Hiwonder/System/app.c and Hiwonder/Peripherals/buzzer.c.

6.1 LED State and Buzzer State Change

```
//按键回调函数
void button_event_callback(ButtonObjectTypeDef *button,  ButtonEventIDEnum event)
{
    if(button == buttons[0]) { /* 按键1的事件  */
        if(event == BUTTON_EVENT_CLICK) { //若为单击
            buzzer_didi(buzzers[0], BUZZER_FREQ, 100, 100, 3);  //蜂鸣器响2声
      static uint8_t led_ctl = 0;
      switch(led_ctl)
      {
        case 0:
          led_off(leds[0]); //LED灭
          led_ctl++;
          break;
        case 1:
          led_on(leds[0]);  //LED亮
          led_ctl++;
          break;
        case 2:
          led_flash(leds[0],200,200,0); //LED闪烁（灭200ms，亮200ms）
          led_ctl = 0;
          break;
        default:
          led_ctl = 0;
          break;
      }
        }
    }
  if(button == buttons[1]) { /* 按键2的事件  */
         if(event == BUTTON_EVENT_DOUBLE_CLICK) { //若为双击
            buzzer_didi(buzzers[0], BUZZER_FREQ, 100, 100, 3);  //蜂鸣器响3声
        }
    }
}
```

Above is the App.c file, where button_event_callback() serves as the button callback function. The main programming flow is as follows:

Button event detection;

Different buzzer sounds for pressing button 1 and button 2;

Different LED state changes based on the number of times button 1 is pressed.

6.2 Buzzer Activation, Deactivation, and Setting of Beep Duration

```
int buzzer_on( BuzzerObjectTypeDef *self, uint32_t freq)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = freq,
        .ticks_on = 1,
        .ticks_off = 0, /* 静音时间为0 长鸣 */
        .repeat = 0,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}

int buzzer_off(BuzzerObjectTypeDef *self)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = 0,
        .ticks_on = 0, /* 鸣响时间为0就是静音 */
        .ticks_off = 0,
        .repeat = 0,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}

int buzzer_didi(BuzzerObjectTypeDef *self, uint32_t freq, uint32_t ticks_on, uint32_t ticks_off, uint32_t repeat)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = freq,
        .ticks_on = ticks_on,
        .ticks_off = ticks_off,
        .repeat = repeat,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}
```

Above is the buzzer.c file, where the buzzer_on() function is to turn on the buzzer. It initializes the buzzer control structure, sets the buzzer frequency, and sets other properties to make the buzzer continuously beep.

.ticks_off = 0, // Silence time is 0, indicating continuous buzzing

The buzzer_off() function is to turn off the buzzer.

.ticks_on = 0, // Beeping time is 0, indicating silence

The buzzer_didi() function is to set the buzzer's frequency and beep mode. It sets the buzzer's frequency, beeping time, silence time, and repeat count.

freq, ticks_on, ticks_off, repeat

6.3 A Callback Function and Button Status Task are Registered

```
118   //注册按键回调函数，处理按键值
119     button_register_callback(buttons[0], button_event_callback);
120     button_register_callback(buttons[1], button_event_callback);
121
```

Above is the app.c file, where the button_register_callback function is used to register a callback function, button_event_callback, for specific buttons (here buttons[0] and buttons[1]). The callback function is invoked when specific events occur, such as pressing or releasing a button. When a button is pressed or released, the callback function button_event_callback is executed, triggering the corresponding code to run.

```
84
85  void button_register_callback(ButtonObjectTypeDef *self, ButtonEventCallbackFuncTypeDef callback)
86  {
87      if(NULL == callback) {
88          return;
89      }
90      self->event_callback = callback;
91  }
92
93  void button_defalut_event_callback(ButtonObjectTypeDef *self,  ButtonEventIDEnum event)
94  {
95  }
96
97  void button_object_init(ButtonObjectTypeDef *self)
98  {
99      self->stage = BUTTON_STAGE_NORMAL;
100     self->last_pin_raw = 0;
101   self->last_pin_filtered = 0;
102     self->combin_counter = 0;
103     self->ticks_count = 0;
104
105     /* config */
106     self->combin_th = 400;
107     self->lp_th = 2000;
108     self->repeat_th = 500;
109     self->event_callback = button_defalut_event_callback;
110 }
```

Above are the functions in the button.c file, including the callback function for registering button events and the default button event callback function.

```c
void button_task_handler(ButtonObjectTypeDef *self, uint32_t period)
{
    self->ticks_count += period;

    uint32_t pin = self->read_pin(self);
    if(pin != self->last_pin_raw)  { /* 前后连续的两次IO状态不同认为按钮状态还不稳定，保存新的IO状态然后返回 */
        self->last_pin_raw = pin;
        return;
    }

    if(self->last_pin_filtered == self->last_pin_raw) { /* 按钮状态没有改变，即状态机状态不会发生转移，直接返回 */
        return;
    }

    self->last_pin_filtered = self->last_pin_raw; /* 保存新的按钮状态 */
    switch(self->stage) {
        case BUTTON_STAGE_NORMAL: {
            if(self->last_pin_filtered) {
                self->event_callback(self, BUTTON_EVENT_PRESSED); /* 触发按键按下事件 */
                if(self->ticks_count < self->combin_th && self->combin_counter > 0) { /* 只有在连击计数不为零时连击才起作用 */
                    self->combin_counter += 1;
                    if(self->combin_counter == 2) {  /* 双击回调 */
                        self->event_callback(self, BUTTON_EVENT_DOUBLE_CLICK);
                    }
                    if(self->combin_counter == 3) {  /* 三连击回调 */
                        self->event_callback(self, BUTTON_EVENT_TRIPLE_CLICK);
                    }
                }
                self->ticks_count = 0;
                self->stage = BUTTON_STAGE_PRESS;
            } else {
                if(self->ticks_count > self->combin_th && self->combin_counter != 0) {
                    self->combin_counter = 0;
```

Above are the functions in the button.c file (partial screenshot omitted). The button_task_handler function handles different button states and events, including detecting button presses, releases, and long presses, and performs corresponding actions as needed.

```c
/**
 * @brief 定时器回调进行按键扫描
 *
 * @param argument
 */
void button_timer_callback(void *argument)
{
  button_task_handler(buttons[0], BUTTON_TASK_PERIOD);
  button_task_handler(buttons[1], BUTTON_TASK_PERIOD);
}
```

The button_porting function periodically scans and updates the real-time status of the buttons.